

LineProof

Project report | 16 September 2026 | Ahsan Maqbool / IZAQ, solo builder

1. Purpose and delivered scope

LineProof is a reproducible robotics verification prototype. It checks a manipulation trajectory, rather than trusting an object ending near its target. The current system demonstrates a scripted single-cube pick, carry, lower and release in MuJoCo, with a separate ACT/OpenVINO runtime preflight. **It is not a completed learned dinner-table robot system.**



2. Why trajectory verification matters

An early endpoint-based scorer accepted an object that had slipped during transfer and landed near its destination. Scorer v3 rejects that recorded failure. A regression test checks both the slipped-object trace and the successful carry trace. The useful contribution is a reproducible failure case, corrected scoring and inspectable evidence; no novelty or industry-first claim is made.

3. Implemented architecture

The scene uses pinned Apache-2.0 SO-101 meshes, two robot arms, two RGB cameras and primitive tabletop props. Twelve finite, bounded joint targets are stepped at a 2 ms physics interval. A scripted IK teacher uses simulator state to control the left arm; the right arm is parked. The object moves through physics contacts, without grasp welds or runtime teleportation. Stop/reset invalidates stale actions. Thirteen repository tests pass.

4. Measured manipulation results

The nominal 19-second simulated trial sustained a minimum lift of 79.54 mm and finished with 9.14 mm horizontal placement error. It retained sampled contacts through carry and released stably. The recording is an actual simulator render, not a physical robot recording or model-driven run.

Seed	Result	Final XY error (mm)
0	PASS	8.78
1	FAIL	34.69
2	PASS	8.68
3	PASS	8.72
4	PASS	8.75
5	FAIL	29.05
6	FAIL	13.05
7	PASS	8.79
8	PASS	8.72
9	FAIL	47.47

Six of ten cube perturbations passed; four failed. Randomization was only cube XY ± 5 mm and yaw ± 0.12 radians. These are scripted local perturbations, not ten full dinner-table variations or learned-policy generalization. Final position alone does not decide success.

5. Scoring and geometry limitations

Every 50 ms sample in hold/transfer must show both jaw contacts, with the cube more than 40 mm above its initial height. Lowering must retain the grasp or show table support near resting height. Final XY error must be below 25 mm, the object must be released, and the last second must be stable within 2 mm. Sampling can miss events between observations; it is not continuous formal verification.

Optional CoACD decomposition replaces only the two jaw collision meshes with twelve convex parts each. Visual geometry, inertias and actuator limits are retained. The hull-count cap prevented the requested concavity tolerance from being achieved: this is an approximation, not a guaranteed 4 mm geometry bound. Generated parts are hash-checked. The 36 tuning reports are exploratory attempts, not an evaluation success rate.

6. Intel runtime measurements

Hardware: Intel i5-1245U laptop, approximately 32 GB RAM, Windows 11. ACT: LeRobot 0.6.0, PyTorch 2.11.0+cpu, OpenVINO 2026.3.1, FP32, four CPU threads. The model has 51,569,548 random parameters, receives two 128x128 RGB images plus 18 state values, and returns ten 12-joint action targets. No model action was executed.

After one warmup, five calls measured 60.634 ms native PyTorch median and 42.649 ms OpenVINO CPU median. Maximum absolute numerical discrepancy was 0.000001386 across the checked fixtures, under the 0.0001 tolerance. Earlier medians differed; this small sample does not establish a production speedup. A separate single synthetic training step took 3.101 seconds with finite gradients, but learned no task and saved no checkpoint.

7. Track readiness and remaining work

The online track expects learned multimodal, multi-step manipulation. No task-trained checkpoint, VLM planner, closed-loop learned action, coordinated dinner-table task, Core Ultra measurement or physical robot test is delivered. Track guidance permits Intel XPU execution beyond Core Ultra, including an explicitly discussed i7 alternative; exact i5-1245U eligibility was not individually confirmed. This submission is partial and may score poorly on task compliance. No winning-readiness claim is made.

Next gate: collect successful demonstrations with the corrected scorer; train a bounded policy on verified available compute; run its actions closed-loop; then evaluate task variations, failures and Intel deployment with a held-out protocol. CUDA setup instructions exist, but the separate Ryzen/RTX 2060 machine has not been tested.

8. Reproduction, evidence and provenance

Repository: <https://github.com/IZAQ18/infra-summit>

Code baseline: 73aeea56ecc08f155c4372474fd8b5c7d8f0ffd8

Follow docs/LOCAL_FEASIBILITY.md for installation and pinned upstream asset fetch. Run the nominal command in docs/CONTACT_EXPERIMENT.md. Run the test suite with `.venv/Scripts/python.exe -m unittest discover -s tests -v`. Reports, state/action traces, the recorded slipped-object counterexample and policy preflight output are in docs/evidence/. Exact package versions, commands and hardware are documented in BENCHMARKS.md.

Project code: MIT. Upstream SO-ARM100/SO-101 assets: Apache-2.0, pinned to eecbe3e0a9ebb23e25ad7b2759b03884c6660903; details and notices in THIRD_PARTY.md. Generated collision parts derive from those assets. No customer data, private account information or invented results are included.

Track clarification source: <https://discord.com/channels/877056448956346408/1547614813977575586/1549440623805989046> (15 September 2026). Additional sources and interpretation: docs/DEPLOYMENT_FALLBACK.md.